

Mit Python arbeiten –
aber kein venv anlegen und aktivieren?

Neue Workflows mit UV

Detlef Lannert <detlef@lannert.de>

PyDdf 10.09.2025

venv gibt es seit 2007

- damals großer Fortschritt: lokale, isolierte Installationen
- kurz danach wurde pip eingeführt
- anschließend undurchsichtige weitere Entwicklungen:
 - setuptools, distutils, distutils2, ...
 - Ergänzungen: pip-tools, virtualenv, virtualenvwrapper, venv-Modul, ...
 - konkurrierende Paketmanager: poetry, hatch, pdm, ...
- seit ca. 2024 existiert uv als neuer Player

„Klassische“ Arbeitsweise

- **immer ein venv anlegen und aktivieren**
- manuelle Auswertung von requirements-Dateien: `pip install -r ...`
- mehrere hiervon nach Bedarf (z.B. dev, prod)
- eigene Workflow-Konventionen
(Updates, Projektversion erhöhen, build, publish)
- ggf. Lockfiles mit pip-compile erstellen
(umgebungs-, plattform-, zweckspezifisch)

Ist uv nur das schnellere pip?

- an eingespielten Workflows hält man gern fest
- viele Blogs empfehlen uv venv, uv pip, uv pip compile
- das funktioniert ganz gut
(viel schneller als pip, ziemlich kompatibel)
- verbessert aber nicht den Workflow
(nur sein Tempo)
- verträgt sich schlecht mit anderen neuen Konzepten

Gewissensfrage:

Will ich überhaupt meine Workflows ändern?

```
if reply != "ja":  
    return
```

Anregungen für neue Workflows

Unsichtbares venv

- kein venv manuell anlegen / aktivieren / verwalten
- nur .venv im ausgecheckten bzw. temporären Projektverzeichnis implizit benutzen
- uv run nutzen (z.B. für mypy)
- mehrfach auschecken, wenn nötig
(gut mit git-Worktree zu verbinden)
- eigenes Tool ggf. userweit installieren
uv tool install .
- Projektumgebung von „außerhalb“ benutzen:
uv run ... --project=\$PROJ_DIR
(oder UV_PROJECT=\$PROJ_DIR setzen)

Versionen managen

- maßgebliche Version nur in pyproject.toml führen
(nicht dynamisch aus einer Datei laden)
- aktualisieren mit uv version --bump=...
- Versionsinfo dem Code zugänglich machen mit

```
# __init__.py
from importlib.metadata import version

__version__ = version("my-great-package")
# Paketname aus der pyproject.toml!
```

Kein venv für Skriptdatei

- wenn das Projekt nur aus einer einzelnen Datei besteht:
→ PEP-723-Metadaten einbetten
- venv wird on the fly erzeugt
- Abhängigkeiten mit Editor oder uv pflegen:
uv add --script myscript.py rich
- Aufruf aus jeder Umgebung möglich

Nur per Wheel installieren

- ... außer zum Entwickeln (editierbares git-Checkout als --editable)
- Staging- oder produktive Umgebung als (temporäres) Projekt anlegen:

```
uv build
cd somewhere_else/
uv init --bare [--python=3.xx] stage
cd stage/
uv python pin 3.xx
uv add --find-links ../dist
uv run my_entry_point
```

Mehrere editierbare Projekte verknüpfen

- dafür kann ein ansonsten leeres Projekt hilfreich sein
- beim (auch editierbaren) Hinzufügen eines Projekts werden dev-Abhängigkeiten jedoch nicht übernommen
- Ausweg: optional dependency statt dependency group

```
uv init --bare --python=3.xx common
cd common/
uv python pin 3.xx
uv add --editable ~/projects/mypkg2[dev]
uv add --editable ~/projects/mypkg1[dev]
uv run mypkg1 check
```

Installation auf Server ohne Developer-Tools oder Internetzugang

Problem:

- einige Pakete sind via PyPI nur als sdist verfügbar
- uv versucht sie bei der Installation zu bauen (wie es pip & Co. auch tun)
- auf dem Server möchte man vielleicht keinen gcc haben
- ... und je nach Netzzugang des Servers können auch keine normalen Wheels aus dem Internet geholt werden
- also braucht man installierbare Wheels, die auf den Server kopiert werden können (rsync)
- uv hat aber kein Äquivalent zu pip wheel

Mögliche Lösung:

- Wheels mit Informationen aus uv.lock herunterladen bzw. bauen
- Experiment: wheel-getter

Anhang: Vergleich der Arbeitsweisen

Die nachfolgende Tabelle stellt einige typische Abläufe für die pip- und die uv-Welt gegenüber, ist allerdings sehr linuxlastig.

	mit pip	mit uv
neue Arbeitsumgebung	<pre>python -m venv --prompt=xyz xyz . venvxyz/bin/activate cd xyz myxyz --foo</pre>	<pre>uv init --bare [--python 3.14] xyz cd xyz [uv python pin 3.14] uv run myxyz --foo</pre>
mit anderer (Python- oder Paket-) Version arbeiten	mehrere parallele venv anlegen	mit uv Versionen wechseln (geht schnell)
in anderes Projekt wechseln	<pre>deactivate cd \$DIR . venv/bin/activate</pre>	<pre>cd \$DIR</pre>
Entwicklungsumgebung von anderem Arbeitsverzeichnis aus benutzen	<pre>\$DIR/venv/bin/python ...</pre>	<pre>uv run --project=\$DIR ...</pre>
Umgebung verlassen	<pre>deactivate cd ...</pre>	<pre>cd ...</pre>
mehrere editierbare Projekte verknüpfen	allgemeine und dev-Requirements der Projekte alle in einem venv (manuell) zusammenfügen und aktivieren	„Arbeitsprojekt“ anlegen, alle lokalen Projekte als editierbar hinzufügen (ggf. mit Extra [dev]), Arbeitsprojekt benutzen durch cd, uv-Option oder Umgebungsvariable